



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Centre for Professional and
Continuing Education

SCTP) Advanced Professional Certificate

AI-Engineering



Object-Oriented Principles Part 2

Lesson Objectives

By the end of this lesson, you will:

- Explain and apply the four OOP pillars in Java
- Use inheritance with extends, super, and protected
- Differentiate method overloading and overriding
- Design abstractions via abstract classes and interfaces
- Model relationships using composition (HAS-A) vs inheritance (IS-A)



Object Oriented Programming

Foundations of OOP



Encapsulation

Bundling data and
hiding
implementation
details



Inheritance

Organizing code in a
parent-child
hierarchy



Polymorphism

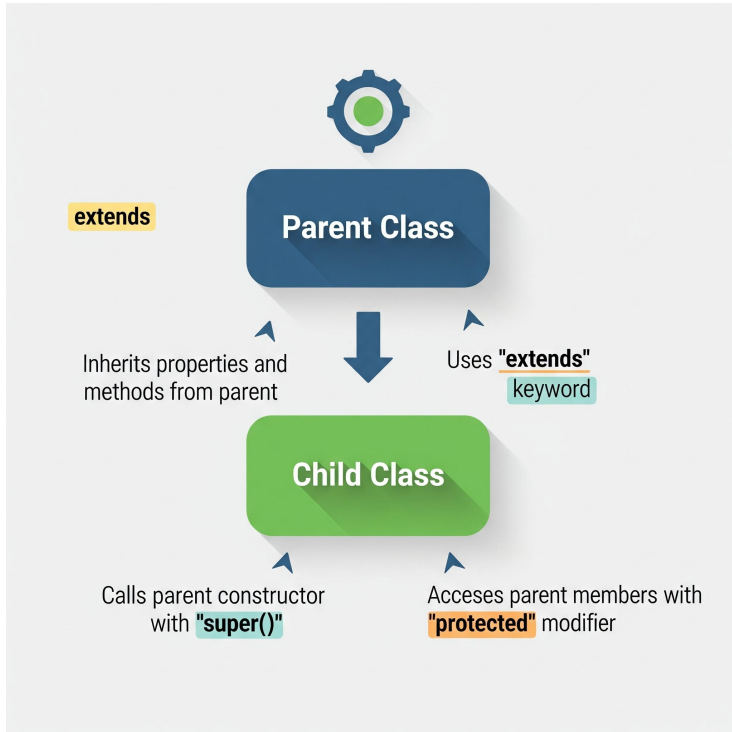
Allowing objects to
take on many forms
with a single
interface



Abstraction

Generalizing
characteristics and
behaviors to simplify
complexity

Part 2: Inheritance



Definition:

- Organizing code into a parent-child hierarchy
- Child inherits properties and methods of parent

Terminology:

- Parent class = Superclass = Base class
- Child class = Subclass = Derived class

Example: Customer → RetailCustomer, CorporateCustomer

Creating a Child Class - RetailCustomer

```
public class RetailCustomer extends Customer {  
    private int accountNumber;  
    private String membershipTier;  
  
    public RetailCustomer(int accountNumber,  
                           String membershipTier) {  
        this.accountNumber = accountNumber;  
        this.membershipTier = membershipTier;  
    }  
}
```

Using extends keyword:

- RetailCustomer extends Customer
- Inherits all fields and methods from Customer
- Can add RetailCustomer-specific fields and methods
- Java does not support multiple inheritance

The super Keyword - Constructor

```
// Calling parent constructor:  
// - Pass name and joinYear to Customer constructor  
// - Initialize RetailCustomer fields separately  
  
public RetailCustomer(String name, int joinYear,  
                      int accountNumber,  
                      String membershipTier) {  
    super(name, joinYear); // Call Customer constructor  
    this.accountNumber = accountNumber;  
    this.membershipTier = membershipTier;  
}
```



What is super?

-  Refers to the parent class (like this for parent)
-  Used to call parent class constructor
-  Java calls super() implicitly if not specified

Rules:

-  super() must be first statement in constructor
-  Cannot call both super() and this() in same constructor



The protected Access Modifier

Access Modifiers:

- **private** – only within class
- **protected** – within class and subclasses
- **public** – accessible from anywhere



When to use protected:

⇒ Allow child classes to access parent fields directly

🔗 Example: `protected String name;`

```
Example:
ff induration:
{
    protected String ();
    protected String_name;
}
```


Polymorphism



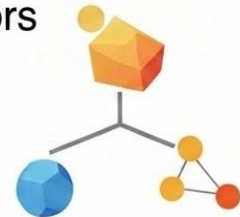
Polymorphism: Key Concepts:

- Polymorphism = Many forms
- Same interface, different behaviors
- Two types: **Compile-time (Method Overloading)** and **Runtime (Method Overriding)**

Definition: - Ability of a method to have different behaviors

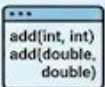
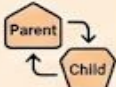
Two Types:

- **Compile-time polymorphism - Method Overloading**
 - Same name, different **parameters**
 - Compiler determines which method to call
- **Runtime polymorphism - Method Overriding**
 - Child overrides **parent** method
 - JVM determines which **method** to call

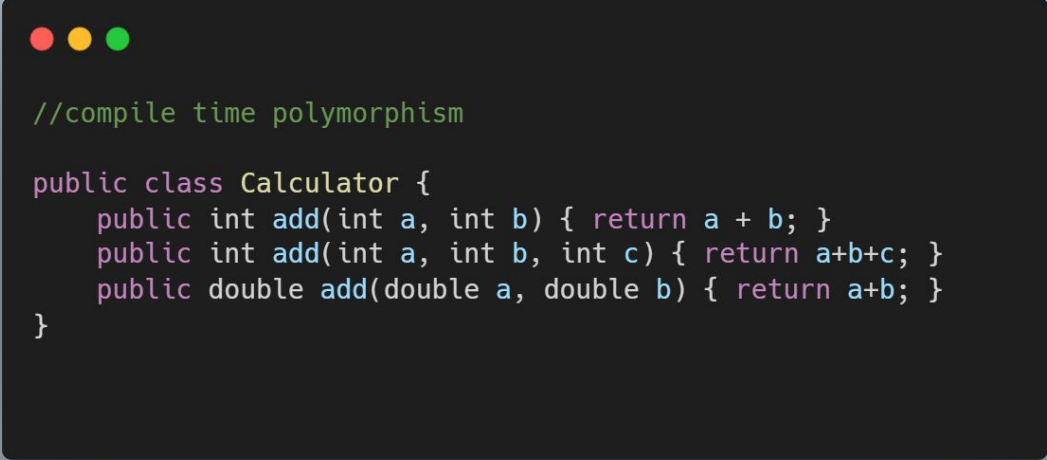


JAVA POLYMORPHISM COMPARISON: COMPILE-TIME vs. RUN-TIME

(Beginner's Guide)

COMPILE-TIME POLYMORPHISM (Static Binding)		VS.	RUN-TIME POLYMORPHISM (Dynamic Binding)	
	When Decided: During compilation phase.			When Decided: During program execution (runtime).
	Java Mechanism: Method Overloading (Same name, different parameters within the same class).			Java Mechanism: Method Overriding (Same signature in Parent & Child classes).
	Performance: Faster execution (no lookup overhead).			Performance: Slower execution (due to dynamic method lookup).
	Example: <pre>class MathUtil { int add(int a, int b) { return a + b; } double add(double a, double b) { return a + b; } }</pre>			Example: <pre>class Animal { void sound() { System.out.println("Animal sound"); } } class Dog extends Animal { @Override void sound() { System.out.println("Bark"); } }</pre>
		Key Takeaway: Compile-time is about 'overloading' and is faster; Run-time is about 'overriding' and offers flexibility.		

Compile-time Polymorphism - Method Overloading



```
//compile time polymorphism
```

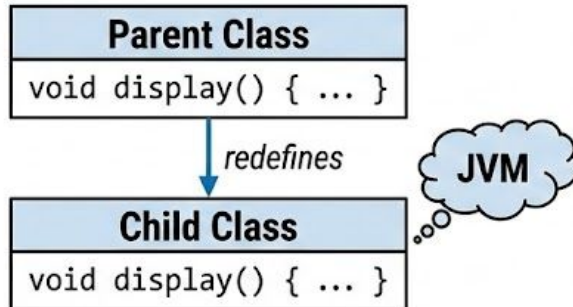
```
public class Calculator {  
    public int add(int a, int b) { return a + b; }  
    public int add(int a, int b, int c) { return a+b+c; }  
    public double add(double a, double b) { return a+b; }  
}
```

Runtime Polymorphism - Method Overriding

METHOD OVERRIDING & THE @OVERRIDE ANNOTATION

► Method Overriding:

- Child class redefines parent method
- Same name and signature as parent
- JVM determines which version to call at runtime



► Why use @Override?



- Indicates method is overriding parent method
- Compiler checks for mistakes
- Without it, typos won't be caught
- Good practice to always use it

```
class Child extends Parent {  
    @Override  
    void display() { ... }  
}
```

Runtime Polymorphism with super

Runtime Polymorphism with super

Calling parent method from child:

Use super to call the parent version

Useful when you want to extend, not replace, behaviour

Parent Class (Customer)

```
public void calculateDiscount() {  
    System.out.println(name + " gets the standard discount.");  
}
```

Child Class (RetailCustomer)

```
@Override  
public void calculateDiscount() {  
    super.calculateDiscount(); // parent runs first  
    System.out.println("Plus an extra 5% retail discount.");  
}
```

— Calls parent's method first

Output:

```
Tony gets the standard discount.  
Plus an extra 5% retail discount.
```


Abstraction

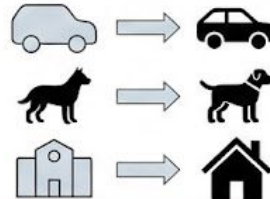
Abstraction

Key Concepts

- Generalizing characteristics and behaviors
- Abstract concepts vs concrete objects
- Two ways to achieve abstraction:
 - Abstract classes 
 - Interfaces 

Definition

- Generalizing a set of characteristics and behaviors
- Abstract concepts: Vehicle, Animal, Building
- Concrete examples: Car, Dog, House



Benefits

- Think about things as groups 
- Generalize functionalities 
- Define contracts for implementation 

Abstract Classes

What is an **abstract class**?

Properties of Abstract Classes

- Declared with abstract keyword
- Can have abstract and non-abstract methods
- Cannot be instantiated
- Used as superclass only

Abstract methods:

- Declared without implementation
- Must be implemented by subclass

Abstract Classes - Customer Example

```
public abstract class Customer {  
    protected String name;  
  
    // Abstract method - no implementation  
    public abstract void calculateDiscount();  
  
    // Non-abstract method  
    public void displayProfile() {  
        System.out.println("Customer: " + name);  
    }  
}  
  
// Cannot instantiate:  
// Customer c = new Customer(...); // ERROR!
```


Interfaces

Understanding Java Interfaces

What is an interface?

- Declared with **interface** keyword
- All methods are **public** and **abstract** by default
- Cannot have instance variables
- Can have default and static methods (Java 8+)

Purpose:

- Define what a class must do
- Not how it should do it

Interfaces - Naming Conventions

1.



Capability/Behavior (suffix: **-able**)

Drivable, Runnable, Trackable

2.



Contract/Type (nouns)

List, Map, Vehicle, Building

3.



Service (operations)

Repository, Service, Handler

Interfaces - Trackable Example

Defining and implementing an interface:

Interface Definition

```
public interface Trackable {  
    void track();  
    // public abstract by default  
}
```

Defines the contract (what)

Implementation in a Class

```
public class MobilePhone  
    implements Trackable {  
    @Override  
    public void track() {  
        System.out.println("Tracking...");  
    }  
}
```

Provides the implementation (how)

Any class can implement Trackable, even classes that are completely unrelated.

Interfaces - Multiple Implementation

- **Key difference from inheritance:**
 - A class can implement multiple interfaces
 - A class can only extend one parent class

Example: Car can be both Trackable and Drivable

```
public class Car implements Trackable, Drivable {  
  
    @Override  
    public void track() { ... }  
  
    @Override  
    public void drive() { ... }  
}
```



Interfaces - Default Methods

- Default methods (Java 8+):
 - Provide default implementation in interface
 - Classes can override or use default
 - Allows adding methods without breaking existing code

```
public interface Drivable {  
    void drive();  
    void stop();  
  
    default void honk() {  
        System.out.println("Honk!");  
    }  
}
```

When to use: Interface vs. Abstract Class

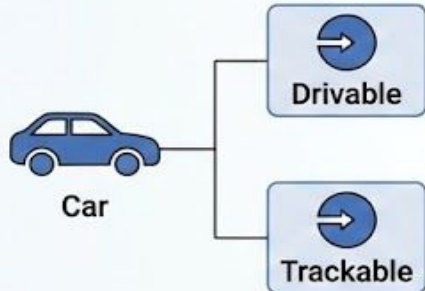
When to use Interface:



- Define contract for any class, regardless of hierarchy

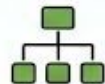


- Need multiple behaviors in a class



Example: Car implements Drivable, Trackable

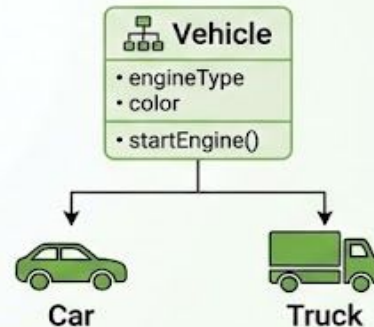
When to use Abstract Class:



- Provide common base for related classes



- Need instance variables or shared methods



Example: Vehicle as base for Car, Truck

Composition: Building Complex Objects

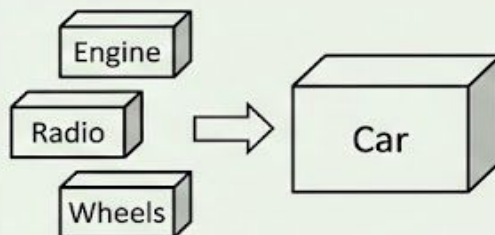
Key Concepts:

- Building classes out of simpler classes 
- **HAS-A** relationship vs **IS-A** relationship (Inheritance)
- Prefer composition over inheritance
- Loosely coupled vs tightly coupled 

What is Composition?

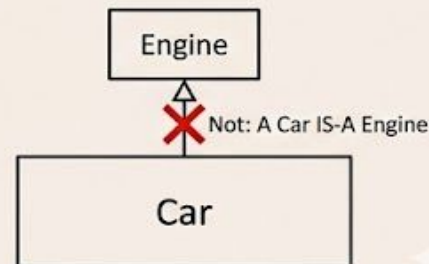
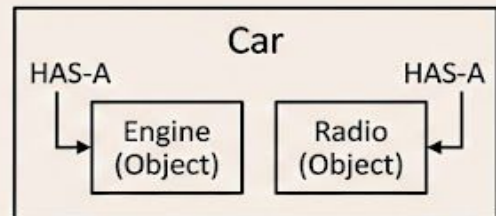
Definition:

- Building a class out of simpler classes
- Combining simple objects into complex ones



Example:

- A Car **HAS-A** Engine, Radio, etc.



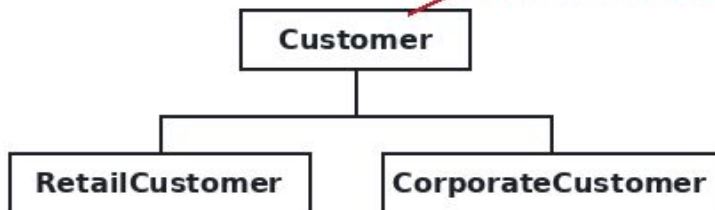
Composition Over Inheritance: Why Prefer Composition?

Inheritance (IS-A)

Tightly coupled relationship

Changes to parent affect all children

Parent change = all children affected



IS-A Examples:

RetailCustomer IS-A Customer

Car IS-A Vehicle

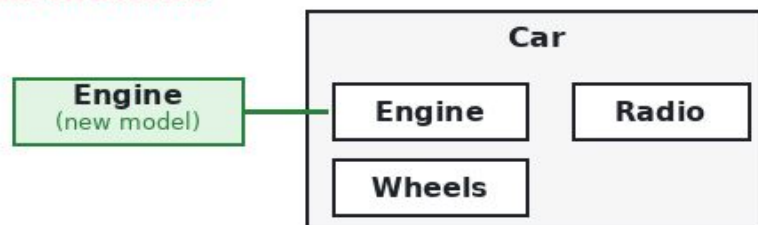
```
class RetailCustomer extends Customer {  
    ...  
}
```

Uses extends keyword

Composition (HAS-A)

Loosely coupled relationship

Changes to composed class don't affect container



Composed change = container unaffected

HAS-A Examples:

Car HAS-A Engine

Car HAS-A Radio

```
class Car {  
    private Radio radio = new Radio();  
}
```

Create instance as field

Composition Example - Radio

Problem:

- Not all vehicles have radio

Solution:

- Use composition instead of inheritance

```
public class Radio {  
    private String model;  
    public void play() { ... }  
}
```



```
public class Car extends Vehicle {  
    private Radio radio;  
  
    public Car(String make) {  
        super(make);  
        this.radio = new Radio("Sony");  
    }  
  
    public void playRadio() {  
        radio.play();  
    }  
}
```



OOP Summary and Beyond

- Encapsulation hides data and exposes only necessary methods
- Inheritance creates parent-child hierarchy using the extends keyword
- Polymorphism allows method overloading and method overriding
- Abstraction is achieved with abstract classes and interfaces
- Composition models HAS-A relationships for flexible design

